

Numerikus Számítások

3. előadás

Csókás Eszter

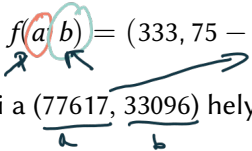
Szegedi Tudományegyetem
Informatika Intézet
Számítógépes Optimalizálási Tanszék



Bevezető

Átvezető példa (és további motiváció)

Tekintsük a következő 2 változót tartalmazó függvényt:

$$f(x) \quad f(a, b) = (333,75 - a^2)b^6 + a^2(11a^2b^2 - 121b^4 - 2) + 5.5b^8 + \frac{a}{2b}$$


és értékeljük ki a $(\underbrace{77617}_a, \underbrace{33096}_b)$ helyen.

Átvezető példa (és további motiváció)

Tekintsük a következő 2 változót tartalmazó függvényt:

$$f(a; b) = (333,75 - a^2)b^6 + a^2(11a^2b^2 - 121b^4 - 2) + 5.5b^8 + \frac{a}{2b}$$

és értékeljük ki a (77617, 33096) helyen.

32-bit: $f = 1.172604$

64-bit: $f = 1.1726039400531786$

128-bit: $f = 1.1726039400531786318588349045201838$

Tehát jók vagyuk?

Átvezető példa (és további motiváció)

Tekintsük a következő 2 változót tartalmazó függvényt:

$$f(a; b) = (333,75 - a^2)b^6 + a^2(11a^2b^2 - 121b^4 - 2) + 5.5b^8 + \frac{a}{2b}$$

és értékeljük ki a (77617, 33096) helyen.

32-bit: $f = 1.172604$

64-bit: $f = 1.1726039400531786$

128-bit: $f = 1.1726039400531786318588349045201838$

Tehát jók vagyuk?

→ a valós érték: $-0.8273960\dots$

Számítások hibái: Kerekítés, levágás

Kerekítési hiba

Láttuk, hogy a számítógép hogyan ábrázolja a számokat és ez sok hibát tud eredményezni: a már jól ismert 0.1 kettes számrendszerben végtelen szakaszos "kettedes" tört, amiből csak az első 52 bitet használjuk, a többit eldobjuk → *ez a véges számábrázolásból eredő kerekítési hiba*

A *kerekítés* szolgál arra, hogy az adott számítógépes környezetben elérhető ábrázolható számok, az ún. gépi számok közül kiválasszuk azt, amellyel eredményünket azonosítjuk.

Az optimális kerekítés az adott számhoz legközelebbi, a számábrázolásban szereplő számot eredményezi.

→ a kerekítési hiba nagyságrendjét a gépi pontossággal lehet megadni: ε

Gépi pontosság

Nézzük meg MATLAB-ban a gépi pontosságot:

```
>> eps
```

```
ans =
```

```
2.2204e-16
```

$2,2204 \cdot 10^{-16}$

```
>> a = 1; b = 12345; c = 1e-14;
```

10^{-14}

```
>> a+c==a
```

```
>> b+c==b
```

Gépi pontosság

Nézzük meg MATLAB-ban a gépi pontosságot:

```
>> eps
```

```
ans =
```

```
2.2204e-16
```

```
>> a = 1; b = 12345; c = 1e-14;
```

```
>> a+c==a
```

```
>> b+c==b
```

```
>> a+c==a
```

```
ans =
```

```
logical
```

```
0
```

```
>> b+c==b
```

```
ans =
```

```
logical
```

```
1
```

Gépi pontosság

A gépi pontosság alapértéke a szám nagyságrendjével változik:

```
>> eps(a)
ans =
2.2204e-16
```

```
>> eps(b)
ans =
1.8190e-12
```

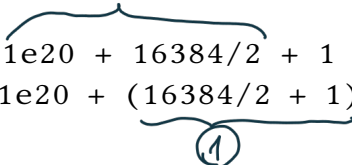
```
>> eps(1e20)
ans =
16384
```

Mit jelent az utolsó példa?

Azt jelenti, hogy 10^{20} után tárolható
”következő” szám 10^{20} -nál 16384-gyel nagyobb!

Akkor mi lehet az eredménye a következőknek?

```
>> 1e20 == 1e20 + 16384/2 + 1
>> 1e20 == 1e20 + (16384/2 + 1)
```



Kerekítési hiba

A levágás:

- egyszerűen elhagyja a jegyek egy részét
- L db jegyre kerekítés során az L -edik utáni jegyeket hagyjuk el
- könnyebb megvalósítani, de kevésbé pontos

Korrekt a kerekítés, ha a szám és a kerekített között nincs gépi szám $\rightarrow$ mind az optimális kerekítés, mind a levágás korrekt kerekítés

Feladat: Optimális kerekítés és levágás ($B = 10, L = 3$)

	optimális	levágás
$x_1 = .123\dot{4}56 \cdot 10^5$	$.123 \cdot 10^5$	$.123 \cdot 10^5$
$x_2 = .567\dot{8}90 \cdot 10^5$	$.568 \cdot 10^5$	$.567 \cdot 10^5$
$x_3 = .123\dot{5}00 \cdot 10^5$	$.124 \cdot 10^5$	$.123 \cdot 10^5$

Alulcsordulás, túlcsordulás

A gyakorlatban az exponens persze véges, és számos probléma adódik abból, hogy az ábrázolandó szám abszolút értéke túl nagy vagy túl kicsi ahhoz, hogy normalizált számmal lehessen reprezentálni. Ekkor beszélünk *felül*-, vagy *alulcsordulás*ról. Az ilyen esetek kezelése a fordítóprogramon múlik. Ha az alulcsordulás miatt denormalizált eredményt kapunk, akkor ez ronthatja a relatív pontosságot. Ezen az előforduló számok nagyságrendjének megváltoztatása, az ún. *skálázás* javíthat.

Nézzük meg a MATLAB-ban:

```
>> int8(-128)-1
```

```
ans =  
int8  
-128
```

```
>> int8(-128)-2
```

```
ans =  
int8  
-128
```

```
>> int8(127)+1
```

```
ans =  
int8  
127
```

Alulcsordulás, túlcsordulás



YouTube
Shared publicly · Dec 1, 2014

We never thought a video would be watched in numbers greater than a 32-bit integer (=2,147,483,647 views), but that was before we met PSY. "Gangnam Style" has been viewed so many times we had to upgrade to a 64-bit integer (9,223,372,036,854,775,808)!

Hover over the counter in PSY's video to see a little math magic and stay tuned for bigger and bigger numbers on YouTube.

PSY - GANGNAM STYLE (강남스타일) M/V

PSY - GANGNAM STYLE (강남스타일) M/V

officialpsy 03

7,889,838

6,761,388

1,648,353

-2142584554

Számítások hibái: Hibaforrások és hibajellemzői

Hibák

Legyen x a keresett, kiszámítandó pontos érték, és legyen $\tilde{x}$ ennek a becslése.

- a *becslés hibája* az $x - \tilde{x}$ érték
- az *abszolút hiba* az $|x - \tilde{x}|$, ami egy nemnegatív szám
- az abszolút hiba egy korlátját abszolút hibakorlátnak nevezzük (több lehetséges van és nyilván azok közül a legkisebb a legjobban használható)
- a hibakorlát pontosságával kapcsolatban:
azt mondjuk, hogy $\tilde{x}$ az x érték egy század pontosságú becslése, ha 0.01 abszolút hibakorlát
- a *relatív hiba* az eltérésnek a pontos érték nagyságrendjéhez való viszonyát adja meg:

$$\frac{|x - \tilde{x}|}{|x|} \leftarrow \frac{\text{absz. hiba}}{|x|}$$

- a kapcsolódó korlát a *relatív hibakorlát*, ami ennek egy felső korlátja (gyakran százalékban vagy ezrelékben adnak meg)

Hibák

Legyen x a keresett, kiszámítandó pontos érték, és legyen $\tilde{x}$ ennek a becslése.

- a *relatív hiba* az eltérésnek a pontos érték nagyságrendjéhez való viszonyát adja meg:

$$\frac{|x - \tilde{x}|}{|x|}$$

- a *pontos tizedesjegyek számát* a relatív hiba negatív tizesalapú logaritmus adja:

$$-\log_{10}\left(\frac{|x - \tilde{x}|}{|x|}\right).$$

Ez persze nem mindig egész szám, de annál pontosabb információt ad.

Példa

 x $\tilde{x}$

Legyen a $\pi = 3.14159265\dots$ és ennek egy becslése 3.141626.

Ekkor a becslés hibája: $x - \tilde{x} = 3,14159265 - 3,141626 = -0,000033$

A becslés abszolút hibája: $|x - \tilde{x}| = 0,000033$

A becslés abszolút hibakorlátja például lehet: 0,000034

A relatív hiba: $\frac{|x - \tilde{x}|}{|x|} = \frac{0,000033}{|3,14159265|} = 0,000010\dots$

A relatív hibakorlát pedig pl.: 0,000011

A pontos tizedesjegyek száma:

$$-\log_{10}\left(\frac{|x - \tilde{x}|}{|x|}\right) = 4,98$$

A hiba forrásai

- Ha már a beolvasott adatok is hibásak, akkor erre vonatkozik az *öröklött hiba*
- A számítási módszer tévedéséből származó eltérés neve *képlethiba*.
- A meghatározott új számok számítógépes ábrázolása során csak véges sok lebegőpontos szám közül választhatunk. Az ekkor elkövetett hiba neve *kerekítési hiba*.

A hibaszámítás alapfeladata

Adjunk hibakorlátot!

De hogyan?

- öröklött hiba és a számítógépes eljárás ismeretében
- az összes lehetséges hibaforrást figyelembe kell venni
- így a feladat nehéz (inkább csak numerikus programkönyvtárak rutinjainak vizsgálata során szokták)

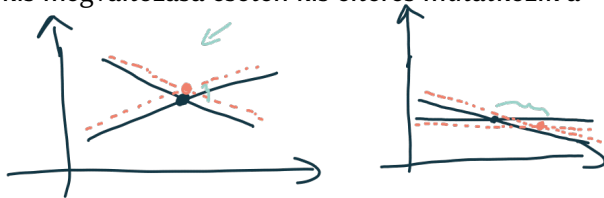
Numerikus stabilitás

Numerikus stabilitás

$$f(a, b)$$

A numerikus stabilitás esetén:

- az illető függvény argumentumának kis megváltozása esetén kis eltérés mutatkozik a függvényértékekben
- ezt is vártuk
- ellentéte a *numerikus instabilitás*



→ célunk olyan numerikus eljárások létrehozása és használata, amelyek a kiszámítandó függvények numerikusan stabilis változatait használják

Kerülendő dolgok numerikus algoritmusok során

- közel azonos számok kivonása
 - ez az eredmény relatív hibájának növekedését eredményezi
 - ilyenkor maga a kivonás viszonylag pontos ugyan, de a korábbi hibák nagyítódnak fel
- kis abszolút értékű számmal való osztást vagy nagy abszolút értékű számmal való szorzás,
 - ez az abszolút hiba növekedését okozza
- általában: kerülni kell az olyan helyzeteket, amikor egy részsámítás eredménye lényegesen nagyobb, vagy kisebb nagyságrendű, mint a végeredmény!

Példa: kivonás veszélye ($B = 10$, $L = 7$ és $L = 6$)

$L = 7$		
1.szám	$0.2789014 \cdot 10^3$	
2.szám	$0.2788876 \cdot 10^3$	
különbség	$0.0000138 \cdot 10^3$	nincs kerekítési hiba
normalizált	$0.1380000 \cdot 10^{-1}$	

$L = 6$	optimálisan kerekített	abszolút hiba	relatív hiba
1.szám	$0.278901 \cdot 10^3$	$0.4 \cdot 10^{-5}$	$1.4342 \cdot 10^{-6}$
2.szám	$0.278888 \cdot 10^3$	$0.4 \cdot 10^{-5}$	$1.4340 \cdot 10^{-6}$
különbség	$0.000013 \cdot 10^3$		nincs kerekítési hiba
normalizált	$0.130000 \cdot 10^{-1}$	$8 \cdot 10^{-4}$	$5.8 \cdot 10^{-2}$

A relatív hiba itt 38ezerszer nagyobb, mint a kezdeti hiba.

Vektorok és mátrixok [(informális) bevezető]

A vektorok (=számok rendezett **sorozatai**) és mátrixok (=számok **táblázatos** elrendezése) a lineáris algebra alapvető eszközei, amelyek adatstruktúrákat és transzformációkat reprezentálnak.

Vektor: n szám $(v_1, v_2, \dots, v_n)$ sorozata, amely lehet sor- vagy oszlopvektor.

Műveletek: összeadás (komponensenként), skalárral való szorzás, belső/skaláris szorzat (eredménye egy szám), külső szorzat (eredménye egy mátrix).

Példa: $n=3$ (v_1, v_2, v_3) $(1, 2, 3) \iff$

$\begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$

oszlop

$\begin{pmatrix} 3 \\ 5 \\ 7 \end{pmatrix}$

sor



A vektorok (=számok rendezett **sorozatai**) és mátrixok (=számok **táblázatos** elrendezése) a lineáris algebra alapvető eszközei, amelyek adatstruktúrákat és transzformációkat reprezentálnak.

Mátrix: m sorból és n oszlopból álló, téglalap alakú számtáblázat (típusa: $m \times n$).

Műveletek: összeadás/kivonás (azonos pozíciójú elemekkel), skalárral szorzás, mátrixszorzás (sor-oszlop szabály), transzponálás (sor-oszlop csere), mátrix inverz (A^{-1}) képzése, determináns számítás ($\det(A)$).

Példa:

$$m=3 \quad n=3 \quad \left| \begin{array}{ccc} 1 & -2 & 3 \\ 5 & 9 & -10 \end{array} \right| \text{ négyzetes}$$

$$m=3 \quad n=4 \quad \left(\begin{array}{cccc} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & -10 & 11 & -12 \end{array} \right)$$

3×4
sor x oszlop táblázat

sol $\left(\begin{array}{ccc} 0 & 4 & 9 \\ & & \end{array} \right)$
overlap

| 1 /